

Day 7 lab: inference runtimes on the Raspberry Pi

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

October 7, 2026

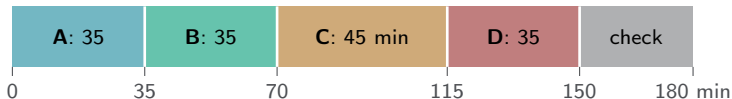
- ▶ **Goal:** your group has a runtime comparison on the Raspberry Pi 5: the latency of one model for each runtime, thread count, and precision.
- ▶ **Board:** the Raspberry Pi 5 with the camera.
- ▶ **Deliverable:** the latency table and the Decision Log in the file `report.md`.

From the lecture to the lab

The lecture gave the reasons: SIMD kernels, threads, the three layers of a runtime, and the graph optimizer. All its numbers are from an x86 processor. In the lab, you export the same model, you look at its graphs, and you measure it on the board.

Key point: Predict each number before you measure it. A wrong prediction teaches more than a correct one.

Plan of the three hours



Part	Work	Time
A	Setup: start the Raspberry Pi, connect with SSH, and test the camera	35 min
B	First inference: MobileNetV2 with LiteRT on a test image and on the camera	35 min
C	Export and inspect: one PyTorch model to ONNX and to LiteRT, and the graphs in Netron	45 min
D	Measure: the latency for each runtime, thread count, and precision	35 min
Check by the instructor		30 min

Write each result in `report.md` when you get it.

Hardware for each group

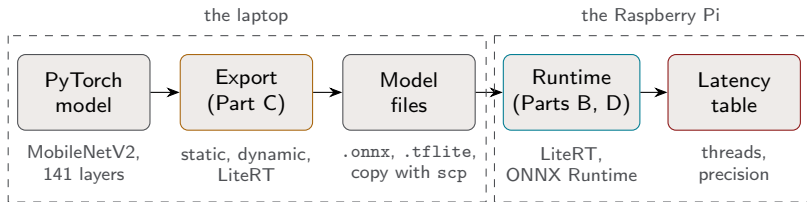
- ▶ Raspberry Pi 5 (8 GB) with the active cooler and the card of your group: pi-NN.
- ▶ Power supply, 27 W.
- ▶ Camera Module 3 with its cable.
- ▶ Laptop in the Wi-Fi edgeai-lab.

Software

- ▶ On the board: the environment `~/tflite_env` with LiteRT and ONNX Runtime.
- ▶ On the laptop: a new environment with the file `requirements.txt`, for the notebook.
- ▶ Netron: `netron.app` in a browser.

Lab files	Labs/day07/ on the laptop, <code>~/edgeai/day07/</code> on the board
Each new terminal on the board	<code>cd ~/edgeai/day07</code> and source <code>~/tflite_env/bin/activate</code>
The camera cable	Do not connect or remove it when the power is on.

Two computers, one model



- ▶ The export needs PyTorch and the converters. They stay on the laptop. The board needs only the two runtimes.
- ▶ The folder `models/` already has three files: the model of the kit lab in `uint8`, and the model of Part C in `int8` for each runtime.
- ▶ You make the `float32` files in Part C and copy them to the board.

Key point: This is the path of each deployment: train and export on a large computer, run a small file on the device.

Step	Work	Time
1	Start the board, and connect with SSH	10 min
2	Check the system with the script of the course card	10 min
3	Test the camera, and look at the photo on the laptop	10 min
4	Copy the lab folder to the board	5 min

```
ssh edge@pi-NN.local          # NN is the number of your group
bash ~/HW-04/check_pi.sh      # each line: PASS, FAIL, or INFO
rpicam-hello --list-cameras
rpicam-jpeg --output test.jpg --width 640 --height 480
scp -r day07 edge@pi-NN.local:~/edgeai/ # on the laptop, in the folder Labs/
```

- ▶ **You record:** the model, the release of the operating system, the Python version, the RAM, and the temperature.
- ▶ Open a second SSH connection and start `htop`. Keep it open: it shows the load of each of the 4 cores.
- ▶ Tell the instructor if a line of the check shows **FAIL**.

Part B: the first inference (35 min)

Get the cat photo of the kit lab, and run the model of the kit lab:

```
wget -P images https://upload.wikimedia.org/wikipedia/commons/3/3a/Cat03.jpg  
python pi/classify_image.py images/Cat03.jpg
```

```
Model:  models/mobilenet_v2_1.0_224_quant.tflite (3577760 bytes)  
Image:  images/Cat03.jpg, threads: 4  
Input:  shape [1, 224, 224, 3], type uint8  
Output: 1001 values, type uint8  
Load: ... ms. First inference: ... ms. Median of the next runs: ... ms.  
Task B1: not complete. The list shows the raw output values.
```

- ▶ The model is MobileNetV2 in uint8, with 1001 classes.
- ▶ The script reads the shape and the type of the input from the file, and prepares the image for them.

Prediction. With one MAC in each clock cycle, this model needs 125 ms on the board. Which median latency do you expect with 4 threads? Write your number in the report. Then run the script.

Source: Photo: Fir0002, Wikimedia Commons.

Step	Work	Time
1, 2	Get the photo, and run the model	15 min
3	Task B1: from the raw outputs to probabilities	10 min
4	Classify photos of the camera, and compare 1 thread with 4 threads	10 min

- ▶ **Task B1** is the function `dequantize_and_softmax` in `pi/classify_image.py`. The mapping of Day 4 gives the real value: $r = S(q - Z)$. Then apply the softmax.
- ▶ The check: the script prints Task B1: complete, and a cat class is first. In the example, the first class has 39 percent.
- ▶ The camera: `python pi/classify_camera.py --count 3 --interval 3`. Point it at a keyboard, a mouse, a cup, or a bottle.
- ▶ **You record:** the five classes of the cat photo, and for each camera photo the object, the first class, and the median latency.

Key point: The file gives the scale and the zero point of its output. Read them from the output details. Do not write them into the code.

Part C: export and inspect (45 min)

Work on the laptop, in the notebook `export_inspect.ipynb`.

Section	Work	Time
0 and 1	Setup, and the layers of the PyTorch model	3 min
2	Task 1: export to ONNX with a static shape	7 min
3	Task 2: export with a dynamic batch size, and a comparison of the two files	8 min
4	Export to LiteRT, and the operators of the file	5 min
5	The graph for each optimization level, and the files in Netron	12 min
6	Task 3: fold a batch normalization by hand	7 min
7 and 8	The same result in each runtime, and the files for the board	3 min

- ▶ The cell after a task prints Task N: complete or not complete.
- ▶ The last cell prints Tasks complete: 1, 2, 3.
- ▶ At the end, copy four files to the board with the `scp` command of section 8.

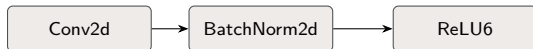
Task	Function	You write
1	<code>export_static</code>	The call of <code>torch.onnx.export</code> , with the names and the options of the notebook
2	<code>export_dynamic</code>	The same call with <code>dynamic_shapes</code> : dimension 0 of the input gets the name <code>batch</code>

- ▶ The checks: the static file has the input shape 1, 3, 224, 224. The dynamic file accepts 3 images. The outputs agree with PyTorch.
- ▶ **You record:** the layers of the PyTorch model, the nodes of the ONNX file, and the layer type that has no node.

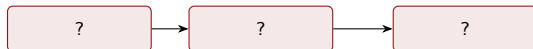
Prediction, before the cell that compares the two files:

- ▶ Is the dynamic file slower for one image?
- ▶ What does the static file do with 2 images?
- ▶ Is a batch of 4 faster for each image than 4 runs with one image?

the PyTorch model



in each file



File in models/

Question of the report

`mnv2_level_basic.onnx`

Which node is gone? Which new input does the first Conv have?

`mnv2_level_extended.onnx`

What is the name of the first node? Which attribute holds the activation?

`mnv2.tflite`

Which property of the first Conv2D holds the activation? Which operator comes before it, and why?

- ▶ Drag a file into the page `netron.app`. Click a node to see its inputs and its attributes.
- ▶ **You record:** the table of the four levels with the nodes and the latency of your laptop. Write your prediction first: which level gives the largest gain?

$$s = \frac{\gamma}{\sqrt{\sigma^2 + \varepsilon}} \quad W' = s W \quad b' = \beta - s \mu$$

- ▶ The function `fold_batch_norm` gets the weight of the first convolution and the four tensors of its batch normalization. It returns W' and b' .
- ▶ The convolution has no bias, so $b = 0$ in the formula of the lecture.
- ▶ Each output channel has its own s . The weight has the shape (output channels, input channels, height, width). Use `s[:, None, None, None]` to multiply each filter by its value.
- ▶ The check runs the two operators of PyTorch, and then one convolution with your W' and b' . The task is complete when the largest difference is below 0.0001.
- ▶ **You record:** the largest difference, and the number of values that the model loses and gets.

Key point: This is the exercise of the lecture, for 32 filters in place of one. The basic level of ONNX Runtime did it for all 52 layers.

Step	Work	Time
1	Predict: write your answer for each question of the report	5 min
2	Task D1: the function <code>median_latency_ms</code> in <code>pi/bench.py</code>	5 min
3	Measure: run the script two times	10 min
4	Fill the latency table, and calculate the three ratios	10 min
5	Compare with your predictions, and write the Decision Log	5 min

runtime	file	type	threads	load ms	median ms
litert	mnv2.tflite	float32	1	...	...
...					
onnxruntime	mnv2_int8.onnx	int8	4	...	...

- ▶ `python pi/bench.py` measures 6 files with 1, 2, and 4 threads: 18 lines. It also writes `results.csv`.
- ▶ Task D1: some calls to warm up, then one time for each call, then the median.

Prediction, before you run the script:

- ▶ `float32`: which runtime is faster, LiteRT or ONNX Runtime?
 - ▶ The gain from 1 thread to 4 threads: 4, more than 3, or less than 3?
 - ▶ LiteRT: is `int8` faster than `float32`? By which factor?
 - ▶ ONNX Runtime: is the dynamic file slower than the static file?
-
- ▶ Close each other program, and keep `htop` open. Use the second run for the table.
 - ▶ **You record**: the latency table, the temperature, the busy cores in `htop`, and three ratios: threads, precision, and runtime.
 - ▶ If you have time: `python pi/bench.py --levels` measures the four optimization levels of ONNX Runtime on the board.

Key point: A latency is valid for one model, one runtime, one setting, and one device. Write all four next to each number.

The instructor checks this at the end of the lab:

- ▶ ✓ `python pi/classify_image.py images/Cat03.jpg` prints Task B1: complete and a cat class as the first class.
- ▶ ✓ The notebook prints Tasks complete: 1, 2, 3 in its last cell.
- ▶ ✓ `python pi/bench.py` prints Task D1: complete.
- ▶ ✓ The latency table of the report has two runtimes or more, two thread counts or more, and two precisions or more.
- ▶ ✓ The report has the five answers of the Netron questions.
- ▶ ✓ The report has a prediction and a result for each question of Part D.
- ▶ ✓ The Decision Log gives numbers and names one trade-off.

Hand in: the file `report.md`.

Write about 100 words. State one design decision, give your measured numbers, and name the trade-off.

A product with a Raspberry Pi 5 and a camera must classify 10 images in each second. The same program also reads the camera and sends each result over the network.

- ▶ Which runtime, which precision, and how many threads do you select for the model?

A good Decision Log:

- ▶ gives the latency of your selection and of one other setting from your table,
- ▶ compares the latency with the budget of 100 ms for each image,
- ▶ names what you lose with your decision. Example: the `int8` file is faster, and it loses about 2 points of accuracy.

If a part does not work, and common problems

Problem	Fix or fallback
The name <code>pi-NN.local</code> does not work	Join the Wi-Fi <code>edgeai-lab</code> . Use the IP address that the instructor gives.
No module named <code>'ai_edge_litert'</code>	The environment is not active: <code>source ~/tflite_env/bin/activate</code>
The camera gives no photo	Do Part B with the cat photo only. Tell the instructor.
The board has no internet connection for the cat photo	Download the photo on the laptop and copy it with <code>scp</code> .
The environment for the notebook does not install	Upgrade <code>pip</code> first. Install only <code>requirements.txt</code> in a new environment. Or work with a second group.
A task is not complete in time	Use the file or the cell of the folder <code>solutions/</code> . Write this in the report.
<code>pi/bench.py</code> prints not found for a file	Copy the file with the <code>scp</code> command of Part C. The instructor has the <code>float32</code> files.
The latency changes much between two runs	Close the other programs. Read the temperature with <code>vcgencmd measure_temp</code> , and wait for one minute.

The README of the lab has the complete list.

- ▶ *Machine Learning Systems* by Vijay Janapa Reddi and contributors, Harvard University. mlsysbook.ai, CC BY-NC-SA 4.0.
- ▶ *Edge AI Engineering: Raspberry Pi* by Marcelo Rovai. github.com/Mjrovai/EdgeML_Made_Ease_ebook and github.com/Mjrovai/EdgeML-with-Raspberry-Pi, GPL-3.0.